

SafeStreet

AI-Powered Community Incident Reporting Platform

Deliverable 4 — AI Integration Report

Web Technologies — Course Project · Intermediate Web Technologies, Full-Stack Track

1. Overview of AI Integration Approach

SafeStreet integrates a Large Language Model (LLM) at four distinct points in the incident lifecycle: classification and department routing at submission time, duplicate detection immediately before a new incident record is created, citizen-facing response drafting when a field worker resolves an incident, and monthly hotspot prediction for the admin analytics dashboard. All four features are implemented as methods on a single NestJS provider, `AiProxyService`, which sits entirely on the backend. No API key, model name, or raw prompt ever reaches the React frontend; the frontend calls ordinary authenticated REST endpoints (e.g. `POST /incidents`, `POST /incidents/:id/draft-response`, `GET /ai/hotspots`) and the NestJS layer is solely responsible for constructing prompts, calling the LLM, and returning structured JSON.

The current implementation uses Google Gemini (model `gemini-flash-latest`) via the `@google/generative-ai` SDK, selected for its low latency, generous free-tier quota for development, and native support for JSON-schema-constrained output (`responseSchema`), which removes an entire class of brittle text-parsing bugs common to free-form LLM output.

Note on provider selection: the course specification lists OpenAI GPT-4o or Anthropic Claude as the required LLM providers. This implementation currently uses Google Gemini instead. This substitution should be confirmed with the course instructor before submission, or the `ai-proxy.service.ts` module should be adapted to call the Anthropic or OpenAI SDK — the prompt templates and control flow documented below would carry over with only the SDK call and response-parsing layer changed.

All four features follow the same architectural pattern: (1) gather minimal, relevant context from PostgreSQL via Prisma, (2) construct a single-shot, stateless prompt with an explicit output schema, (3) call the model, (4) parse and validate the structured response, and (5) fall back to a deterministic, non-AI default if any step fails. This pattern is detailed per-feature in Section 2, with the cross-cutting context management strategy discussed in Section 3.

2. Prompt Templates, Inputs, and Outputs per AI Feature

2.1 AI Feature 1 — Incident Classifier & Router

Method: `AiProxyService.classifyAndRoute()`. Called synchronously during `POST /incidents`, before the incident record is persisted.

Input	Citizen-submitted title, free-text description, optional address, and latitude/longitude coordinates.
Output schema	category (enum), departmentCode (string), priority (enum: LOW/MEDIUM/HIGH/CRITICAL), rationale (string), confidence (number).
Structured output	Enforced via Gemini <code>responseSchema</code> (<code>SchemaType.OBJECT</code>) — the model cannot return free text; malformed output is impossible by construction, only missing/failed calls are possible.

Prompt template (exact, as implemented):

```
You are a municipal incident triage assistant for "SafeStreet". A citizen
has
submitted the following report. Classify it accurately and route it to the
correct department.

Title: {title}
Description: {description}
Address: {address, or "Not provided"}
Coordinates: {latitude}, {longitude}

Guidance:
- CRITICAL priority = immediate danger to life (e.g. exposed live wires,
  deep
  open manholes, structural collapse risk).
- HIGH priority = significant safety/health risk but not immediately
  life-threatening (e.g. large pothole on a busy road, major water leak).
- MEDIUM priority = standard maintenance issue (e.g. broken streetlight,
  minor
  pothole).
- LOW priority = cosmetic/non-urgent (e.g. graffiti, litter).
- departmentCode should be one of: ROADS, SANITATION, WATER, ELECTRICAL,
  PARKS, PUBLIC_SAFETY, OTHER.
- rationale should be a concise (1-2 sentence) explanation a supervisor can
  quickly review.

Respond ONLY with the JSON object matching the provided schema.
```

Figure 2.1 — Verbatim prompt template used in `classifyAndRoute()`. Curly-brace placeholders are interpolated from the request body at call time.

Fallback / degradation behaviour:

If the Gemini call throws (network failure, quota exceeded, malformed response), the method catches the error and returns a deterministic safe default rather than failing the citizen's submission:

```
{
  category: "OTHER",
  departmentCode: "OTHER",
  priority: "MEDIUM",
  rationale: "AI classification unavailable at time of submission -
    routed to general queue for manual triage.",
  confidence: 0
}
```

This routes the incident to the General/Unclassified department queue for manual supervisor triage, satisfying the course requirement that at least one AI feature gracefully degrade when the AI service is unavailable — in this implementation, all four features degrade gracefully by design, not just one.

2.2 AI Feature 2 — Duplicate Detector

Method: `AiProxyService.detectDuplicates()`, calling the private helper `compareWithGemini()`. Called synchronously during `POST /incidents`, immediately after classification and before the record is created.

Input	New report's title, description, coordinates, and category, plus up to 20 existing open incidents within a computed bounding box.
Pre-filtering	Two-stage: (1) a cheap indexed SQL range query on latitude/longitude bounding box, (2) an exact Haversine great-circle distance calculation in application code to strictly enforce the 50-metre radius, discarding any bounding-box false positives near the corners.
Output schema	matches: array of { incidentId, similarityScore (0.0–1.0), reason }, one entry per candidate incident.
Decision rule	isDuplicate = true when the highest similarityScore exceeds 0.75, matching the course-specified threshold.

Prompt template (exact, as implemented):

```
A new incident report was submitted within 50 meters of the following
EXISTING
open incidents. For each existing incident, score from 0.0 to 1.0 how likely
it describes the SAME real-world problem as the new report (1.0 = certainly
the same issue, 0.0 = clearly unrelated despite proximity).

NEW REPORT:
Title: {title}
Description: {description}

EXISTING NEARBY INCIDENTS:
- id: "{id}" | title: "{title}" | description: "{description}"
  ... (one line per candidate, up to 20)

Respond ONLY with JSON matching the schema:
{ "matches": [{ "incidentId", "similarityScore", "reason" }, ...] }
one entry per existing incident listed above.
```

Figure 2.2 — Verbatim prompt template used in `compareWithGemini()`. Only geographically-filtered candidates are included, keeping the candidate list short and relevant.

Fallback / degradation behaviour:

If the Gemini semantic comparison call fails, the system does not skip duplicate detection entirely — it falls back to proximity-only matching, flagging every candidate already within the 50-metre radius at a fixed moderate score (0.5) with the reason "Proximity-based match only (AI comparison unavailable)". This score sits below the 0.75 auto-flag threshold, so a human supervisor reviewing open incidents will still see the geographic overlap without the AI's judgement asserting a false duplicate.

2.3 AI Feature 3 — Citizen Response Drafter

Method: AiProxyService.draftResponse(). Called on demand via POST /incidents/:id/draft-response when a field worker is resolving an incident.

Input	Incident title, incident category, and the free-text resolution notes typed by the field worker.
Output schema	{ draft: string } — a single string field.
Constraint	Prompt explicitly requests exactly 3 sentences and instructs the model not to invent details beyond what the worker provided, reducing hallucination risk in citizen-facing communication.

Prompt template (exact, as implemented):

```
Write a short, warm, and empathetic message to a citizen letting them know their reported issue has been addressed. Use EXACTLY 3 sentences. Do not invent details beyond what's given. Sign off simply as "The SafeStreet Team".

Incident: {incidentTitle} ({incidentCategory})
Resolution notes from field worker: {resolutionNotes}

Respond ONLY with JSON: { "draft": "..."
```

Figure 2.3 — Verbatim prompt template used in draftResponse(). The worker reviews and can freely edit the returned draft before it is saved, via the AIFeedback component.

Fallback / degradation behaviour:

If the call fails, a deterministic template string is generated in code rather than an AI-authored sentence: "Thank you for reporting this issue. Our team has completed work on "{incidentTitle}": {resolutionNotes}. We appreciate you helping keep our community safe. - The SafeStreet Team". The worker still sees and can edit this before it is sent, so the citizen always receives a coherent message even during an AI outage.

2.4 AI Feature 4 (Bonus) — Hotspot Predictor

Method: AiProxyService.predictHotspots(). Called on demand via GET /ai/hotspots from the Admin Analytics tab.

Input to aggregation step	All incidents created within the last 90 days (configurable), each with latitude, longitude, and category.
Aggregation (non-AI)	Incidents are bucketed in application code into a ~0.005°(≈500m) grid; each cell accumulates a count, an average coordinate, and a per-category tally. This is deterministic arithmetic, not an LLM call.

Input to LLM step	Only the top 5 grid cells by incident count (coordinates, count, dominant category) — never the raw per-incident text.
Output	A grid array (all cells, for map rendering) plus a 2–3 sentence natural-language summary generated by the LLM purely from the aggregated numbers.

Prompt template (exact, as implemented):

```
Here are the top incident hotspot grid cells from the last {daysBack} days
(lat, lng, incidentCount, dominantCategory):
({lat}, {lng}) count={incidentCount} category={dominantCategory}
... (up to 5 lines, one per top cell)

Write a 2-3 sentence plain-English summary for a city supervisor
highlighting
the most concerning patterns and what department(s) should prioritize
attention.
```

Figure 2.4 — Verbatim prompt template used in `predictHotspots()`. Deliberately given only pre-aggregated counts, not free text, keeping the request small, fast, and cheap regardless of how many raw incidents exist.

Fallback / degradation behaviour:

The grid itself is always computed and returned regardless of AI availability, since it is pure arithmetic. Only the narrative summary is AI-generated; if that call fails, the summary field defaults to the static string "Hotspot analysis complete.", and the grid (and therefore the heatmap visualization on the admin map) is entirely unaffected.

3. Context Management Strategy

SafeStreet’s AI integration deliberately avoids maintaining conversational/chat history or a persistent context window per user. Every call to `AiProxyService` is a single, stateless request-response cycle. This decision was made for four concrete reasons specific to this domain:

- Each feature answers a narrow, well-defined question (classify this one report / compare this one report against these N candidates / draft a message for this one resolution / summarise these 5 grid cells) that does not benefit from prior turns.
- Statelessness keeps token usage — and therefore latency and cost — proportional to a single incident’s data, not to a growing conversation history, which matters at municipal scale (the brief cites ~3,200 reports/month).
- It removes an entire class of bugs and privacy concerns around session/context leakage between unrelated citizens’ reports.
- It makes every AI call independently retryable and cacheable in principle, since output depends only on the current request’s inputs.

3.1 How context is assembled per call

Rather than passing raw database rows to the model, each method performs an application-level context-reduction step before constructing its prompt:

- Classifier: only the fields a human triager would actually read (title, description, address, coordinates) are included — no internal IDs, timestamps, or unrelated incident data.
- Duplicate detector: candidates are pre-filtered by SQL bounding box and exact Haversine distance before ever reaching the LLM, so the model reasons over perhaps 1–20 geographically-relevant incidents rather than the entire incidents table.
- Response drafter: only the specific incident's own title, category, and the worker's own resolution notes are included — no other citizens' data is ever in scope for this call.
- Hotspot predictor: raw incident rows are aggregated into grid cells in JavaScript first; the LLM only ever sees five summarised rows of numbers, never the underlying free-text descriptions or any per-incident personal data.

3.2 Structured output as a context/parsing strategy

All four features (except the hotspot narrative summary, which is deliberately open-ended prose) use Gemini's responseSchema feature to constrain output to a specific JSON shape with typed enums. This is treated as part of context management because it removes ambiguity from the "context" the rest of the application must interpret: downstream code (NestJS services, WebSocket events, React components) can trust the shape of the AI's response without defensive parsing, regex extraction, or prompt-injection-driven format drift.

4. Prompt Iterations and Rationale

The two iterations below document the reasoning behind the final prompt design choices for the Classifier/Router and Duplicate Detector features. Teams should verify this section reflects their own actual development history and adjust the specifics (dates, exact wording tried) to match what was genuinely attempted, per the course's academic integrity requirement that prompt documentation be original team work.

4.1 Iteration — Incident Classifier & Router

Iteration 1 (initial approach): free-text output, parsed with string matching

The first working version asked Gemini to respond in a loosely-specified format, e.g. "Category: <x>\nDepartment: <y>\nPriority: <z>\nReason: <text>", and the NestJS service split the response on newlines and colons to extract each field.

Problems observed: the model would occasionally add conversational preamble ("Sure, here is the classification:"), use different casing or punctuation for field labels, or wrap the priority value in a full sentence, all of which broke the naive string-splitting parser intermittently and unpredictably — exactly the kind of non-deterministic failure that is hard to reproduce and debug.

Iteration 2 (final approach): JSON-schema-constrained output

The prompt was rewritten to request JSON explicitly, and the Gemini call was configured with `responseMimeType: "application/json"` and an explicit `responseSchema` enumerating the exact fields, types, and allowed enum values for category and priority (Section 2.1).

Rationale for the change: schema-constrained output moves format correctness from "hoped-for LLM behaviour" to "enforced by the API layer", eliminating parser fragility entirely. It also allowed the `required[]` list in the schema to guarantee every field the rest of the application depends on (category, departmentCode, priority, rationale, confidence) is always present, so downstream code never needs null-checks for a missing AI field — only a try/catch around the call itself for outright failures.

4.2 Iteration — Duplicate Detector

Iteration 1 (initial approach): AI-only distance judgement

An early design considered asking the LLM to judge both geographic proximity and topical similarity in a single prompt, passing it the new report plus a broad set of "recent" incidents without any pre-filtering by distance.

Problems anticipated: this couples two very different kinds of reasoning (precise geometric distance vs. fuzzy semantic similarity) into one prompt, is far more token-expensive as the incident table grows, and makes the 50-metre threshold specified in the course brief an implicit, unverifiable part of the model's judgement rather than an explicit, auditable rule.

Iteration 2 (final approach): hybrid deterministic-filter + AI-semantic-compare pipeline

The final design separates concerns: exact distance filtering (bounding box + Haversine) is done deterministically in application code, and the LLM is only asked the question it is actually well-suited to answer — "do these two pieces of free text describe the same real-world problem?" — over a short, geographically-guaranteed-relevant candidate list.

Rationale for the change: this keeps the 50-metre radius as a hard, testable, non-negotiable constraint enforced by code (satisfying the course's explicit requirement), uses the LLM only for the fuzzy judgement it is suited to, and bounds token usage to at most 20 candidates regardless of total incident volume.

5. Data Privacy Implications

Each AI call transmits citizen-authored free text (incident titles and descriptions), geographic coordinates, and — for the response drafter — a field worker's resolution notes, to Google's Gemini API over HTTPS. No account credentials, email addresses, phone numbers, or internal user IDs are included in any prompt; the context-reduction described in Section 3.1 was deliberately designed so that only the minimum incident-specific text and location data needed for each task leaves the backend, and other citizens' unrelated data is never in scope for a given call. That said, incident descriptions are free text a citizen could choose to include personal details in (e.g. "my neighbor at [address] told me..."), and precise GPS coordinates are inherently sensitive location data even without a name attached, since

repeated reports from the same citizen could in principle be correlated by location. Google's API terms and data-retention/training policies for the Gemini API should be reviewed and disclosed to end users (e.g. in a privacy policy) before this system handles real citizen data in production, and — as noted in Section 2 — the current provider (Gemini) differs from the course's specified options (GPT-4o/Claude), each of which carries its own distinct data-handling terms that would need equivalent review.

End of Deliverable 4 — AI Integration Report.